Übung 1 (GIT)

Abschlussbedingungen

In diesem Aufgabenzettel geht es um die praktische Nutzung von GIT.

Aufgabe 1

Vorbereitung:

- Generieren Sie einen Schlüssel und laden Sie diesen nach git.thm.de hoch (siehe Vorlesung).
  Kopieren Sie Ihren **öffentlichen** Schlüssel (Endung: .pub), nicht den privaten!

- Setzen Sie die globalen Attribute (siehe Vorlesung).

- Installieren Sie die "git bash".

- Erstellen Sie auf git.thm.de ein neues Projekt "Testprojekt".

- Clonen Sie dieses auf Ihrem Rechner und wechseln Sie in der git Bash in dieses Verzeichnis.

Aufgabe 2

Führen Sie in der git bash folgende Aktionen durch:

a) Legen sie eine neue Datei a.txt an: `touch a.txt`
Verifizieren Sie mit `cat a.txt` , dass die Datei leer ist.

b) Fügen Sie diese Datei Ihrem Index hinzu und setzen Sie den HEAD auf diesen neuen Stand.

c) Lassen Sie sich den log anzeigen.

d) Fügen Sie eine Zeile in die Datei ein und speichern diese neue Version.

e) Zeigen Sie den Unterschied zwischen der neuen und der vorigen Version an.

f) Ermitteln Sie die Nummer des vorigen Commits und setzen Sie den HEAD auf diesen zurück. Verifizieren Sie, dass die Datei leer ist.

Aufgabe 3 (Git Issues)

a) Öffnen Sie ein bestehendes Git-Projekt oder legen Sie ein neues an. Legen Sie eine Markdown-Datei mit beliebigem Inhalt darin an und comitten Sie diese.

b) Erlauben Sie unter Settings -> General -> Visibility das Anlegen von Issues.
Jetzt sollten Sie links einen neuen Menüpunkt "Plan" sehen.

c) Legen Sie ein Issue "Markdown anpassen" an und weisen Sie es sich selbst zu.
Wo sehen Sie jetzt das neue Issue?
"Bearbeiten" Sie das Issue und schließen Sie es.
Wie können Sie sich das nun geschlossene Issue noch ansehen?

d) Legen Sie ein weiteres Issue an, weisen es sich selbst zu. Vor dem Schließen sollen Sie einen neuen Branch anlegen (wo geht das beim Bearbeiten des Issues?), die Datei ändern und einen Merge Request erstellen.
Approven Sie den Merge Request und führen Sie ihn durch. Welchen Status hat das Issue danach?

Übung 2 (BS)

Abschlussbedingungen

Aufgabe 0

Wenn Sie Windows nutzen, installieren Sie bitte WSL2 (https://learn.microsoft.com/de-de/windows/wsl/install) und Windows Terminal.

Aufgabe 1

Starten Sie Windows Terminal und darin Ubuntu oder (unter MacOS oder Linux) ein Terminalfenster.

1. Legen Sie in Ihrem Home-Verzeichnis ein Unterverzeichnis PI2TEST an und wechseln Sie in dieses Verzeichnis. Geben Sie den Namen des aktuellen Verzeichnisses aus.
2. Legen Sie mit dem Kommando `touch dummy` eine neue Datei an. Welche Berechtigungen hat diese Datei und wie groß ist sie?
3. Kopieren Sie die Datei und löschen Sie die Originaldatei. Verschieben Sie die neue Datei in Ihr Home-Verzeichnis.
4. Wechseln Sie in Ihr Home-Verzeichnis und verifizieren Sie, dass die neue Datei dort ist.

Aufgabe 2

Die Datei a.txt habe (unter Linux) die Permissions -rw-r--r--

1. Geben Sie das Kommando an, um allen Benutzern die Rechte zu geben, die Datei zu ändern. Geben Sie dabei die numerische und die symbolische Schreibweise an.
2. Geben sie ein(!) Kommando an, um allen Benutzern Lese- und Schreibrechte auf der Datei zu entziehen und gleichzeitig dem Eigentümer Rechte zu geben, die Datei auszuführen.
   Geben Sie auch hier die numerische und symbolische Notation an.

Tipp: Mit dem Kommando `man` erhalten Sie mehr Information zu Kommandos.

Aufgabe 3

1. Lassen Sie sich die Liste aller Ihrer Prozesse anzeigen.
2. Lassen Sie sich die Manual Page des Kommandos ps anzeigen. Suchen Sie darin, wie man nur die Prozesse anzeigt, deren Vaterprozess die Nummer 1 hat.
3. Probieren Sie das Kommando `top` aus.
4. Unterbrechen Sie `top`, indem Sie Strg-Z drücken. Welchen Status hat der Prozess jetzt?
5. Nutzen Sie die Kommandos `jobs` und `fg`, um den Zustand abzufragen und zu ändern.

Aufgabe 4:

Recherchieren Sie per KI den Begriff "Round-Robin".

1. Identifizieren Sie die Vorlesungsfolien für deren Inhalte "Round-Robin" relevant ist.
2. Sollte Ihnen die KI ein Beispiel zeigen, versuchen Sie diese Schritt-für-Schritt nachzuvollziehen, ansonsten fragen Sie die KI nach einem Beispiel.
3. Welche Prozessübergänge (siehe Folie"Prozesszustände") sind für "Round-Robin" relevant?
4. Reflektion:
    - Sind die Antworten korrekt und vollständig? (Wie evaluieren Sie die Qualität der Antworten?)
    - Gibt es Widersprüche oder Unklarheiten in den Antworten?

Übung 2 - Lösung

Abschlussbedingungen

Aufgabe 1

1. uwe@DESKTOP-0O7TS1G:~$ mkdir PI2TEST
   uwe@DESKTOP-0O7TS1G:~$ cd PI2TEST/
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ pwd
   /home/uwe/PI2TEST

2. uwe@DESKTOP-0O7TS1G:~/PI2TEST$ touch dummy
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l
   total 0
   -rw-r--r-- 1 uwe uwe 0 Jun 10 17:24 dummy
   Die Datei ist für den Eigentümer (uwe) les- und schreibbar, für alle anderen nur lesbar. Die Datei ist 0 Byte groß.

3. uwe@DESKTOP-0O7TS1G:~/PI2TEST$ cp dummy dummy-neu
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ rm dummy
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ mv dummy-neu ..
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls
4. uwe@DESKTOP-0O7TS1G:~/PI2TEST$ cd ..
   uwe@DESKTOP-0O7TS1G:~$ ls -l dummy-neu
   -rw-r--r-- 1 uwe uwe 0 Jun 10 17:25 dummy-neu

Aufgabe 2

1. uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l a.txt
   -rw-r--r-- 1 uwe uwe 0 Jun 10 17:28 a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ chmod og+w a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l a.txt
   -rw-rw-rw- 1 uwe uwe 0 Jun 10 17:28 a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ chmod 666 a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l a.txt
   -rw-rw-rw- 1 uwe uwe 0 Jun 10 17:28 a.txt

2. uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l a.txt
   -rw-rw-rw- 1 uwe uwe 0 Jun 10 17:28 a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ chmod o-rw,u+x a.txt
   uwe@DESKTOP-0O7TS1G:~/PI2TEST$ ls -l a.txt
   -rwxrw---- 1 uwe uwe 0 Jun 10 17:28 a.txt

   alternativ: chmod 760 a.txt

Aufgabe 3

1. ps x
   Zeigt alle Prozesse des aktuellen Users inklusive der Hintergrundprozesse.

Aufgabe 4

- Das Round Robin Verfahren ist relevant für Kapitel 3b (Prozesse).
- Die Prozesszustände, für die Round Robin relevant ist, sind
  "Ready" : Der Prozess ist jetzt in der "Round Robin-Liste".
  "Running": Der Prozess war an der Reihe und hat jetzt die CPU; nach Ablauf der Zeitscheibe geht der Prozess wieder in "Ready"
  Prozesse, die "Waiting" sind, sind zwar in der Liste, werden aber übergangen.

Übung 3 (Express)

Abschlussbedingungen

Aufgabe 1

Heute bauen wir das WebService-Beispiel aus der Vorlesung nach. Bitte wenden Sie die Schritte der Reihe nach an und melden Sie sich, sollten Fehler auftreten.

1. Starten Sie WebStorms
2. Legen Sie ein neues Projekt des Typs "Express" an.
   Wählen Sie unter 'Location' einen geeigneten Ort und Namen für Ihr Projekt.
   Wählen Sie 'View Engine'=none und 'Stylesheet Engine'=Plain CSS.
   'Node Interpreter' und 'express-generator' sollten gefüllt sein.
   Nach 'Create' sollten Sie ein Projekt haben, welches schon einen Ordner 'public' und andere hat.
3. Erzeugen Sie eine 'tsconfig.json' Datei - wissen Sie noch, wie das geht?
   Verifizieren Sie bitte, dass in der Datei "esModuleInterop" auf "true" gesetzt ist.
4. Setzen Sie für TypeScript "Recompile on Changes" - wissen Sie noch wo?
5. Installieren Sie die Node-Typen für TypeScript. Dazu öffnen Sie ein Terminal (z.B. in WebStorm ganz unten) und geben ein:
   `npm install -D @types/node`
   `npm install -D @types/express`
   Damit werden dem TypeScript-Compiler die Typen aller in Node und Express vordefinierten Funktionen
   bekannt gemacht. Ohne diese lässt sich die folgende TypeScript-Datei nicht kompilieren.
6. Installieren Sie Express:
   `npm install express`
7. Kopieren Sie drei Dateien adder.html, adderFetch.ts und adder.ts (siehe Moodle-Kurs) und legen Sie sie im Projekt wie folgt ab:
   adder.html -> 'public';
   adderFetch.ts -> 'public/javascripts'
   adder.ts -> in den Hauptordner.
8. Starten Sie 'adder.js'
   Sie sollten eine Meldung 'Server running on port 3000' sehen.
9. Öffnen Sie im Browser die folgende URL:
   http://localhost:3000/adder.html

Aufgabe 2

Die Übungsleiter werden Ihnen eine kleine Beispielanwendung zeigen. Diskutieren Sie in kleinen Gruppen, welche Bestandteile notwendig sind und bauen Sie die Anwendung nach. Die Gruppe, die die beste und vollständigste Lösung hat, bekommt einen kleinen Preis.

Adder Beispiel

Abschlussbedingungen

adder.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Title</title>
   <script src="javascripts/adderFetch.js"></script>
</head>
<body>
<div class="container">
   <h1>Adder</h1>
   <form id="add-form">
       <div class="input-group">
           <span class="input-group-text">a</span>
           <input type="text" id="a" class="form-control">
           <span class="input-group-text">b</span>
           <input type="text" id="b" class="form-control">
           <span class="input-group-text">c</span>
           <input type="text" id="c" class="form-control">
           <button class="btn btn-outline-secondary" type="submit"> =
</button>
           <span class="input-group-text" id="result">?</span>
       </div>
   </form>
</div>

</body>
</html>
```

---

adderFetch.ts

```
document.addEventListener('DOMContentLoaded', () => {
   document.getElementById('add-form')?.addEventListener('submit', async
(event) => {
       event.preventDefault();
       const a = (document.getElementById('a') as HTMLInputElement).value;
       const b = (document.getElementById('b') as HTMLInputElement).value;
       const c = (document.getElementById('c') as HTMLInputElement).value;

       const res = await fetch('add', {
           method: 'post',
           headers: {
               "Content-type": "application/json"
           },
           body: JSON.stringify({a, b, c})
       });

       const data = await res.json();
       console.log(res.status + ' ' + res.statusText);
       if (res.ok) {
           const resul = document.getElementById('result');
           if (resul) {
               resul.textContent = data.sum;
           }
```

```
        }
    });
});
```

---

adder.ts

```
import express, { Express, Request, Response } from 'express';

const app: Express = express();
const port:number = 3000;

app.use(express.json());

app.use("/", express.static(__dirname+'/public'));

app.listen(port, () => { console.log('Server running on port: ' + port)});

app.post("/add", (req: Request, res: Response) => {
   let reply: number = Number(req.body.a) + Number(req.body.b) +
Number(req.body.c);
   console.log("Ergebnis = " + reply);
   res.setHeader('Content-Type', 'application/json');
   res.status(200)
       .end(JSON.stringify({sum:reply}));
});
```

Übung 4 (Vert.X)

Abschlussbedingungen

In der Übung der letzten Woche haben Sie das Adder-Beispiel mit Express, also den Server mit TypeScript umgesetzt. Heute geht es darum, den Server-Teil in Java mit Vert.X zu implementieren.
Bitte nutzen Sie dazu den Generator auf https://start.vertx.io/ und verwenden Sie die Group Id “de.thm.mni.pi2” und die Artefact-ID "adder"; wählen Sie bei "Show dependencies panel" die Option Vertx Web aus und generieren Sie das Projekt.

1)

a) Entpacken Sie das Projekt (zip-file) und starten öffnen Sie es als Projekt mit IntelliJ oder öffnen Sie den Ordner mit VisualStudio Code.

b) Führen Sie das MainVerticle aus (siehe Anleitung auf Folie "Eclipse Vert.X"). Rufen Sie die Route, die das MainVerticle anbietet, im Browser auf (welche URL müssen Sie verwenden?).

c) Legen Sie ein "AdderVerticle" an, indem Sie die gleiche Route /add implementieren wie in dem Express-Server, Ermöglichen Sie auch, dass man adder.html über Ihren Server aufrufen kann.
Testen Sie mit der adder.html Seite der letzten Woche.

2)

a) Fügen Sie eine Route /count hinzu, die die Anzahl der Aufrufe der Route /add zurückgibt.

b) Erstellen Sie eine Route, die z.B. bei Aufruf GET /monat/11 die Antwort "November" zurückliefert.

c) Recherchieren Sie, wie man CORS in VERT.x erlaubt.

Übung 4 (Musterlösung)

Abschlussbedingungen

AdderVerticle.java

```
package de.thm.mni.pi2.adder;

import io.vertx.core.Future;
import io.vertx.core.VerticleBase;
import io.vertx.core.http.HttpMethod;
import io.vertx.core.json.JsonObject;
import io.vertx.ext.web.Router;
import io.vertx.ext.web.RoutingContext;
import io.vertx.ext.web.handler.BodyHandler;
import io.vertx.ext.web.handler.CorsHandler;
import io.vertx.ext.web.handler.StaticHandler;

import java.time.Month;
import java.time.format.TextStyle;
import java.util.*;

public class AdderVerticle extends VerticleBase {
 int counter = 0;

 @Override
 public Future<?> start() {
   int port = 8888;

   Router router = Router.router(vertx);
   // Allow CORS for all origins
    router.route().handler(CorsHandler.create()
      .allowedMethod(HttpMethod.GET)
      .allowedMethod(HttpMethod.POST)
      .allowedMethod(HttpMethod.PUT)
      .allowedMethod(HttpMethod.DELETE)
      .allowedHeader("Content-Type")
      .allowedHeader("Authorization"));

   router.post("/add")
     .handler(BodyHandler.create()).handler(this::add);

   router.get("/counter").handler(rc -> {
      rc.response()
        .setStatusCode(200)
        .putHeader("content-type", "text/plain")
        .end("Anzahl Aufrufe: " + counter);
   });
```

```
  router.route("/static/*").handler(
    StaticHandler.create("public"));

  router.get("/monat/:nummer").handler(rc -> {
    int nummer = Integer.parseInt(rc.request().getParam("nummer"));
    String monat = switch (nummer) {
      case 1 -> "Januar";
      case 2 -> "Februar";
      case 3 -> "März";
      case 4 -> "April";
      case 5 -> "Mai";
      case 6 -> "Juni";
      case 7 -> "Juli";
      case 8 -> "August";
      case 9 -> "September";
      case 10 -> "Oktober";
      case 11 -> "November";
      case 12 -> "Dezember";
      default -> "Invalid Month";
    };

    // Alternativ:
    // monat = Month.of(nummer).getDisplayName(TextStyle.FULL, Locale.GERMAN);
    rc.response().setStatusCode(200).end(monat );
  });

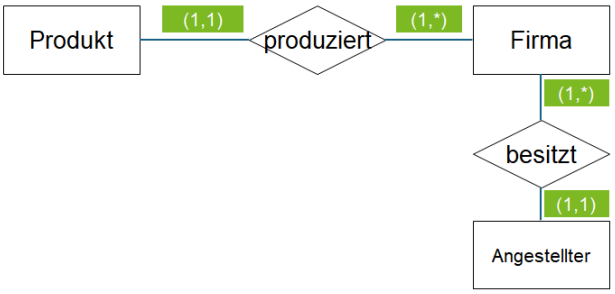
  return vertx
    .createHttpServer()
    .requestHandler(router)
    .listen(port).onSuccess(http -> {
      System.out.println("HTTP server started on port 8888");
    });
 }

 private void add(RoutingContext rc) {
  try {
    counter++;
    JsonObject jsonBody = rc.body().asJsonObject();
    int a = jsonBody.getInteger("a");
    int b = jsonBody.getInteger("b");
    int c = jsonBody.getInteger("c");
    JsonObject ergebnis = new JsonObject().put("result", a + b + c);
    rc.response().setStatusCode(200)
      .putHeader("content-type", "application/json")
      .end(ergebnis.encode());
  } catch (Exception e) { rc.response().setStatusCode(400).end("Ungültiges JSON");  }
 }
}
```

Übung 5

Übung 5

1) Beschreiben Sie verbal das folgende Datenmodell.



2) Modellieren Sie folgenden Sachverhalt:

a) Ein Geschäftsbereich (einer Firma) betreut mehrere Kunden.
Was sind sinnvolle Attribute der jeweiligen Entitäten?

b) Ein Kunde zieht um - wie können Sie die Historie der Adressen abbilden?

c) Durch den Umzug ist nun ein neuer Geschäftsbereich für den Kunden zuständig. Wie können Sie die Historie der Betreuungen modellieren?

3) Modellieren Sie:
Eine Person hat einen Namen und ein Geburtsdatum. Personen haben einen Vater (auch eine Person). Väter haben Kinder.

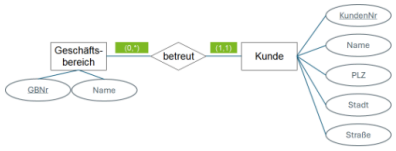
Übung 5 (Lösung)

1)

- Eine Firma produziert n Produkte
- Ein Produkt wird von genau einer Firma produziert
- Eine Firma besitzt n Angestellte
- Jede(r) Angestellte arbeitet genau für eine Firma

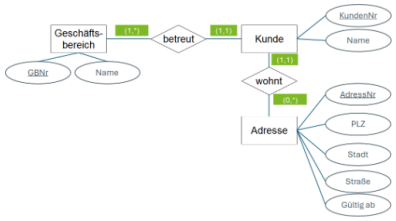
2)

a)



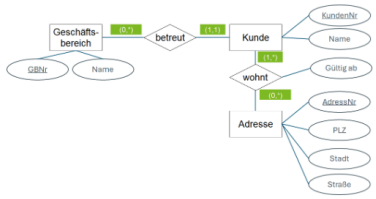
Hier stellt sich die Frage, was passiert, wenn zwei Kunden die gleiche Adresse haben. Um redundante Adressen zu vermeiden, sollte man die Adressen als separate Entität speichern.

b) Mit einer neuen Entität "Adresse" mit zusätzlichem Attribut würde sich folgendes Modell ergeben:



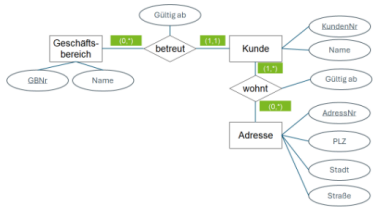
Nun stellt sich aber die Frage, was passiert, wenn zwei Kunden zunächst an der gleichen Adresse wohnen, einer aber umzieht: Ändert sich dann für beide "Gültig ab"? Ist die Kardinalität (1,1) von "Kunde" zu "wohnt" noch richtig

Hier das verbesserte Modell:



c)

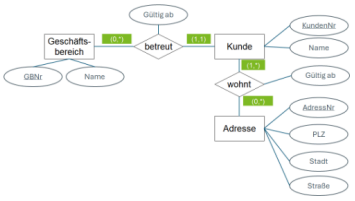
Analog fügen wir ein Attribut "gültig ab" an das Betreuungsverhältnis an.





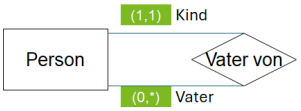
c)

Analog fügen wir ein Attribut "gültig ab" an das Betreuungsverhältnis an.



3)

Eine Person ist Kind von genau einem Vater und eine Person ist Vater von keiner oder beliebig vielen Kindern.



Zuletzt geändert: Montag, 24. November 2025, 08:19

Übung 6

Abschlussbedingungen

Stellen Sie sicher, dass Sie die Tabellen "Dozent" und "Kurs" mit dem ersten Skript angelegt und gefüllt haben.
Unterhalb dieser Übung finden Sie noch ein zweites Skript, das sie bitte auch einlesen.

1) SELECT

a)

Sie wollen die Namen der Studierende, den Namen der von ihnen absolvierten Module und ihre Note ausgeben lassen. Warum führt `select name, modulname, rating from students, kurs, ratings;` nicht zum richtigen Ergebnis?

b) Geben Sie die korrekte Abfrage an.

c) Selektieren Sie alle Kurse, bei denen Mueller mehr als 75 Punkte erreicht hat.

d) Geben Sie die Anzahl der Studierenden aus.

e) Was ist an folgender Abfrage überflüssig?
```
select s.name, r.rating, k.modulnr from students s, ratings r, kurs k where
s.matrikelnr=r.sid and r.kid=k.modulnr;
```

f) Ermitteln Sie die Namen der Studierenden, die genau zwei Kurse absolviert haben. Benutzen Sie dabei nicht COUNT.

2) Unterabfragen

Unterabfragen (subqueries) kann man auf verschiedene Arten verwenden:

a) In Where-Klauseln:
Benutzen Sie den Operator IN, um die Namen aller Studierenden zu ermitteln, deren Matrikelnummer in der Tabelle ratings vorkommt.

b) In Select's:
Beispiel: `SELECT column1, (SELECT column2 FROM other_table WHERE condition) AS subquery_result FROM table_name;`

Geben Sie für jeden Studierenden aus, wieviele Kurse er/sie belegen könnte (also wieviele Kurse es insgesamt gibt).

c) In FROM-Klauseln
Beispiel: `select avg(erg) from (select rating/100 as erg from ratings) as derived_table;`

Was berechnet diese Abfrage? Wie kann man sie umformulieren?

d) Mit der Bedingung EXISTS (Unterabfrage) kann man die Tupel abfragen, die in der Ergebnismenge der Unterabfrage enthalten sind. Recherchieren Sie zu EXISTS und schreiben Sie eine Abfrage, die ermittelt, welche Studierenden INF1005 absolviert haben.

Skript 2_p2_students_ratings.sql

```
   Abschlussbedingungen
use pi2;

create table students (
    matrikelnr int primary key,
    name varchar(64)
);

insert into students values (5551111, "Mueller");
insert into students values (5552222, "Mayer");
insert into students values (5553344, "Schmidt");
insert into students values (5555566, "Schulze");

create table ratings (
  sid int,
  kid varchar(7),
  rating int,
  FOREIGN KEY (sid) REFERENCES students(matrikelnr),
  FOREIGN KEY (kid) REFERENCES kurs(ModulNr)
);

insert into ratings values (5551111, "II1001", 80);
insert into ratings values (5551111, "INF1003", 70);
insert into ratings values (5552222, "INF1003", 60);
insert into ratings values (5552222, "INF1005", 75);
insert into ratings values (5553344, "INF1005", 95);
insert into ratings values (5555566, "TI5005", 80);
```

Übung 6 Musterlösung

Abschlussbedingungen

Stellen Sie sicher, dass Sie Tabellen "Dozent" und "Kurs" mit dem Skript angelegt und gefüllt haben.
Unterhalb dieser Übung finden Sie noch ein zweites Skript, das sie bitte auch einlesen.

1) SELECT

a)

Sie wollen die Namen der Studierenden, die Namen der von ihnen absolvierten Module und ihre Note ausgeben lassen. Warum führt `select name, modulname, rating from students, kurs, ratings;` nicht zum richtigen Ergebnis?

Antwort: Ohne die Bedingungen, die die Primär- - Sekundärschlüsselbeziehungen abbilden, werden die Tabellen lediglich "ausmultipliziert".

b)

Geben Sie die korrekte Abfrage an.

Antwort:

```
select name, modulname, rating from students s, kurs k, ratings r where
r.sid=s.matrikelnr and k.modulnr=r.kid;
```

c)

Selektieren Sie alle Kurse, bei denen Mueller mehr als 75 Punkte erreicht hat.

Antwort:

```
select modulname from kurs k, ratings r, students s where s.name="Mueller"
and r.sid=s.matrikelnr and r.kid=k.modulnr and r.rating>75;
```

d)

Geben Sie ddie Anzahl der Studierenden aus.

Antwort:

```
select count(*) from students;
```

e)

Was ist an folgender Abfrage überflüssig?

```
select s.name, r.rating, k.modulnr from students s, ratings r, kurs k where
s.matrikelnr=r.sid and r.kid=k.modulnr;
```

Antwort:

Die Modulnummer kann hier auch direkt aus der Ratings-Tabelle gelesen werden, dann entfällt auch der Abgleich mit der Kurs-Tabelle:

```
select s.name, r.rating, r.kid from students s, ratings r where
s.matrikelnr=r.sid;
```

Hinweis: Die Verknüpfung mehrerer Tabellen kostet immer Zeit und Speicherplatz. Versuchen Sie, solche überflüssigen Verbünde zu vermeiden!

f) Ermitteln Sie die Namen der Studierenden, die genau zwei Kurse absolviert haben. Benutzen Sie dabei nicht COUNT.

Antwort:

```
select distinct s.name from students s, ratings r1, ratings r2 where
(r1.sid=s.matrikelnr and r2.sid=matrikelnr and r1.kid<>r2.kid);
```

2) Unterabfragen

Unterabfragen (subqueries) kann man auf verschiedene Arten verwenden:

a) In Where-Klauseln:
Benutzen Sie den Operator IN, um die Namen aller Studierenden zu ermitteln, deren Matrikelnummer in der Tabelle ratings vorkommt.

Antwort:

```
select s.name from students s where s.matrikelnr in (select sid from
ratings );
```

b) In Select's:
Beispiel: `SELECT column1, (SELECT column2 FROM other_table WHERE condition) AS subquery_result FROM table_name;`

Geben Sie für jeden Studierenden aus, wieviele Kurse er/sie belegen könnte (also wieviele Kurse es insgesamt gibt).

Antwort:

```
select s.name, (select count(*) from kurs) from students s;
```

c) In FROM-Klauseln
Beispiel: `select avg(erg) from (select rating/100 as erg from ratings) as derived_table;`
Was berechnet diese Abfrage?

Antwort: Die Abfrage berechnet den Durchschnitt aller ratings in Prozent.

d) Mit der Bedingung EXISTS (Unterabfrage) kann man die Tupel abfragen, die in der Ergebnismenge der Unterabfrage enthalten sind. Recherchieren Sie zu EXISTS und schreiben Sie eine Abfrage, die ermittelt, welche Studierenden INF1005 absolviert haben.

Antwort:

```
select s.name from students s where exists (select * from ratings r where
r.sid=s.matrikelnr and r.kid="INF1005");
```

Übung 7

Abschlussbedingungen

Diese Aufgaben setzen wieder auf den Tabellen students, kurs, ratings aus den vorigen Übungen auf.

Aufgabe 1 (join und JDBC)

a) Nutzen Sie einen join, um die Liste der Studierenden mit Namen und mit ihren Noten pro Modul auszugeben.

b) Setzen Sie a) mit JDBC um. Nutzen Sie ein Prepare-Statement!

c) Benutzen Sie den folgenden Code, um den ersten Parameter der Kommandozeile abzufragen. Wenn Sie den Namen eines Studierenden eingeben, sollen sein Name, die Modulnummern und Noten seiner Kurse angezeigt werden.

```
String StudentName="Mueller";
if(args.length > 0) {
    StudentName = args[0];
}
```

Aufgabe 2 (Selbststudium)

a) Nutzen Sie einen join, um die Liste der Studierenden mit ihren Durchschnittsnoten auszugeben.

b) Fügen Sie "Hoppenstedt", Matrikelnummer 5511990 zur Tabelle students hinzu.
Wiederholen Sie 3a) mit einem LEFT JOIN, RIGHT JOIN. Wie unterscheiden sich die Ergebnisse?

c) Ermitteln Sie für alle Studierenden deren durchschnittliche Bewertung.

d) Ermitteln Sie für jeden Kurs dessen durchschnittliche Bewertung.

e) Geben Sie die Namen der Studierenden aus, der durchschnittliche Bewertung größer gleich 80 ist.

f) Geben Sie zu jedem Studierenden aus, wieviele Kurse er/sie absolviert hat.

Ubung 7 (Lösung)

Abschlussbedingungen

StudentNotenAusgabe.java (Aufgabe 1)

```
package de.thm.mni.pi2.database;

import java.sql.*;

/**
* Main-Klasse zur Ausgabe der Studierenden mit ihren Noten
* Verwendet JDBC mit PreparedStatement für einen JOIN-Query
*/
public class StudentNotenAusgabe {

   private static final String DB_URL = "jdbc:mysql://localhost:3306/pi2";
   private static final String DB_USER = "user1";
   private static final String DB_PASSWORD = "passwort";

   public static void main(String[] args) {
       String StudentName="Mueller";
       if(args.length > 0) {
               StudentName = args[0];
       }
       StudentNotenAusgabe studentNotenAusgabe = new StudentNotenAusgabe();

       studentNotenAusgabe.zeigeStudentenMitNoten();
       studentNotenAusgabe.zeigeNotenFuerStudent(StudentName);
   }

   /**
    * Zeigt alle Studierenden mit ihren Noten an
    * Verwendet einen JOIN über die Tabellen students, ratings und kurs
    */
   public void zeigeStudentenMitNoten() {
       String sql = "select s.name, k.modulname, r.rating " +
                    "from students s " +
                    "join ratings r on s.matrikelnr=r.sid " +
                    "join kurs k on r.kid=k.modulnr";
       executeQuery(sql, "=== Liste der Studierenden mit ihren Noten ===",
null);
   }

   /**
    * Zeigt die Noten für einen bestimmten Studierenden an
    * Verwendet einen JOIN mit WHERE-Klausel und
PreparedStatement-Parameter
    * @param studentName Name des Studierenden
    */
   public void zeigeNotenFuerStudent(String studentName) {
       String sql = "select s.name, k.modulname, r.rating " +
```

```
                    "from students s " +
                    "join ratings r on s.matrikelnr=r.sid " +
                    "join kurs k on r.kid=k.modulnr " +
                    "where s.name = ?";
        executeQuery(sql, "=== Noten für Student: " + studentName + " ===",
studentName);
    }

    /**
     * Führt eine SQL-Abfrage aus und gibt die Ergebnisse formatiert aus
     * @param sql SQL-Query mit optionalem Parameter (?)
     * @param ueberschrift Überschrift für die Ausgabe
     * @param studentName Optional: Name des Studierenden für WHERE-Klausel
     */
    private void executeQuery(String sql, String ueberschrift, String
studentName) {
        try (Connection connection = DriverManager.getConnection(DB_URL,
DB_USER, DB_PASSWORD);
             PreparedStatement preparedStatement =
connection.prepareStatement(sql)) {

            // Parameter setzen, falls vorhanden
            if (studentName != null) {
                preparedStatement.setString(1, studentName);
            }

            try (ResultSet resultSet = preparedStatement.executeQuery()) {
                System.out.println("\n" + ueberschrift + "\n");
                System.out.printf("%-20s %-35s %-6s%n",
                    "Name", "ModulName", "Rating");

System.out.println("----------------------------------------------------------
-------------------------");

                boolean hasResults = false;
                while (resultSet.next()) {
                    hasResults = true;
                    String name = resultSet.getString("name");
                    String modulName = resultSet.getString("modulname");
                    int rating = resultSet.getInt("rating");

                    System.out.printf("%-20s %-35s %-6d%n",
                        name, modulName, rating);
                }

                if (!hasResults) {
                    String fehlermeldung = studentName != null
                        ? "Keine Daten für Student '" + studentName + "'
gefunden."
                        : "Keine Daten gefunden.";
                    System.out.println(fehlermeldung);
                }
            }
```

```
       } catch (SQLException e) {
           System.err.println("Fehler beim Ausführen der
Datenbankabfrage:");
           System.err.println("SQL State: " + e.getSQLState());
           System.err.println("Error Code: " + e.getErrorCode());
           System.err.println("Message: " + e.getMessage());
           e.printStackTrace();
       }
   }
}
```

2a) select s.name, avg(rating) from students s, ratings r where s.matrikelnr=r.sid group by s.matrikelnr;
2b) select s.name, avg(rating) from students s left join ratings r on s.matrikelnr=r.sid group by s.matrikelnr;
Hier wird Hoppenstedt mit NULL ausgegeben.

2d) select k.modulname, avg(rating) from kurs k, students s, ratings r where s.matrikelnr=r.sid and r.kid=k.modulnr group by k.modulnr;

2e) select s.name, avg(rating) from students s join ratings r on s.matrikelnr=r.sid group by s.matrikelnr having avg(rating)>80;

2f) select s.name, count(modulnr) from students s join ratings r on s.matrikelnr=r.sid join kurs k on r.kid=k.modulnr group by s.matrikelnr;

Übung 8

Abschlussbedingungen

Aufgabe 1

a) Informieren Sie sich auf https://www.toptal.com/developers/bcrypt/ über das BCrypt API. Probieren Sie das Generierieren und Prüfen von salted hashes aus.

b) Als Ausgabe nach generate hash erhalten Sie einen langen String. Recherchieren Sie, was die Bestandteile dieses Strings sind.

c) Erstellen Sie eine kleine Java-Anwendung, die nach Eingabe eines initialen Passwortes und des Stärkefaktors mit dem BCrypt API einen Hash erstellt. Dabei können Sie sich an dem Beispiel aus der Vorlesung orientieren.

d) Recherchieren Sie, welche Möglichkeiten das API bietet, um ein Passwort gegen einen Hash zu prüfen.
Erweitern Sie Ihre Anwendung so, dass der Benutzer solange nach einem Passwort gefragt wird (Tipp: Scanner.nextLine()), bis dieses zu dem initialen Passwort passt.

Übung 8 (Musterlösung)

Abschlussbedingungen

```
package org.example;

import org.mindrot.jbcrypt.BCrypt;
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
       String salt;
       String hashed;
       String password="";
       int strength=0;
       Scanner scanner = new Scanner(System.in);

       if (args.length==2) {
               password = args[0];
               strength = Integer.parseInt(args[1]);
           }
       else {
           System.err.println("Please provide password and strength");
           System.exit(0);
       }

       salt = BCrypt.gensalt(strength);
       hashed = BCrypt.hashpw(password, salt);
       System.out.println("Salt= " + salt + ", hash= " + hashed );
```

```
        while (true) {
            System.out.print("Enter Password > ");
            String input = scanner.nextLine();
            if (BCrypt.checkpw(input, hashed)) {
                System.out.println("Password Matched!");
                break;
            } else {
                System.out.println("Password Incorrect!");
            }
        }
    }
}
```